INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Práctica 3: Text Representation

NATURAL LANGUAGE PROCESSING

7CM2

INTEGRANTES:

VÁZQUEZ BLANCAS CESAR SAID

PROFESOR:

JORGE LUIS ROSAS TRIGUEROS

FECHA DE ENTREGA: 17 DE MARZO DE 2026

FECHA DE REALIZACION DE LA PRACTICA: 14 DE MARZO DE 2026

Marco Teórico

El Procesamiento de Lenguaje Natural (Natural Language Processing, NLP) es una rama de la inteligencia artificial que se enfoca en el desarrollo de métodos computacionales capaces de analizar, interpretar y generar lenguaje humano de forma automática. Esta disciplina combina conocimientos de lingüística, estadística, aprendizaje automático y ciencias de la computación para permitir que las computadoras trabajen con grandes volúmenes de información textual. Entre sus aplicaciones más relevantes se encuentran los motores de búsqueda, sistemas de recomendación, análisis de sentimientos, traducción automática, sistemas de diálogo, clasificación de documentos y detección de similitud entre textos (Jurafsky & Martin, 2023).

Uno de los principales desafíos del procesamiento de lenguaje natural es que el lenguaje humano está compuesto por estructuras complejas, ambigüedades semánticas y múltiples variaciones lingüísticas. Para que un sistema computacional pueda analizar textos de manera efectiva, es necesario transformar la información textual en una representación matemática que permita aplicar algoritmos de análisis y comparación. Este proceso se conoce como representación de texto y constituye una etapa fundamental en el análisis automático de documentos.

Antes de representar los textos numéricamente, se realiza un proceso de preprocesamiento del lenguaje natural, cuyo objetivo es limpiar y normalizar los datos textuales para mejorar la calidad del análisis posterior. El preprocesamiento permite eliminar elementos irrelevantes, reducir la variabilidad del lenguaje y facilitar la identificación de patrones dentro del corpus de documentos.

Una de las técnicas básicas dentro del preprocesamiento es la tokenización, que consiste en dividir el texto en unidades más pequeñas llamadas *tokens*. Generalmente los tokens corresponden a palabras individuales, aunque en algunos casos también pueden representar frases o subpalabras. La tokenización permite que el texto pueda ser analizado de manera estructurada por los algoritmos de procesamiento de lenguaje natural (Manning, Raghavan & Schütze, 2008).

Otra etapa importante es la normalización del texto, la cual incluye operaciones como la conversión de todas las palabras a minúsculas, la eliminación de signos de puntuación y la eliminación de caracteres especiales. Estas transformaciones permiten reducir la variabilidad de las palabras y evitar que diferentes representaciones de un mismo término sean interpretadas como palabras distintas por los algoritmos de análisis.

Dentro del preprocesamiento también se realiza la eliminación de stopwords, que son palabras muy frecuentes en un idioma pero que generalmente no aportan información relevante sobre el contenido del texto. Ejemplos de stopwords en inglés incluyen palabras como *the*, *is*, *and*, *of*, entre muchas otras. Debido a su alta frecuencia en la mayoría de los

documentos, estas palabras suelen tener poco valor discriminativo para tareas de análisis de texto, por lo que su eliminación contribuye a mejorar la eficiencia y precisión de los modelos (Aggarwal, 2018).

Otra técnica importante en el preprocesamiento es la lematización, que consiste en reducir las palabras a su forma base o forma canónica conocida como *lema*. Por ejemplo, palabras como *running*, *runs* y *ran* pueden reducirse al lema *run*. Este proceso permite agrupar diferentes variaciones morfológicas de una palabra bajo una misma representación, lo que facilita el análisis del contenido semántico del texto y reduce la dimensionalidad del vocabulario utilizado en el análisis.

Una vez que el texto ha sido preprocesado, es necesario transformarlo en una representación numérica que permita su análisis computacional. Una de las técnicas más utilizadas para este propósito es TF-IDF (Term Frequency – Inverse Document Frequency). Esta técnica permite calcular la importancia de cada término dentro de un documento en relación con un conjunto de documentos conocido como *corpus*. TF-IDF se basa en dos medidas principales: la frecuencia del término dentro del documento y la rareza del término dentro del conjunto de documentos.

La primera medida, conocida como Term Frequency (TF), calcula cuántas veces aparece una palabra dentro de un documento. En términos generales, una mayor frecuencia indica que la palabra tiene mayor relevancia dentro de ese documento. Sin embargo, algunas palabras pueden aparecer frecuentemente en muchos documentos sin aportar información significativa sobre el contenido del texto.

Para solucionar este problema se utiliza la segunda medida, conocida como Inverse Document Frequency (IDF). Esta medida calcula qué tan rara o poco frecuente es una palabra dentro de todo el corpus de documentos. Las palabras que aparecen en muchos documentos reciben un peso menor, mientras que aquellas que aparecen en pocos documentos reciben un peso mayor. De esta forma, TF-IDF permite identificar términos que son representativos del contenido específico de cada documento (Salton & Buckley, 1988).

Al combinar ambas medidas, TF-IDF genera una representación vectorial de los documentos en la cual cada dimensión del vector corresponde a una palabra del vocabulario del corpus. De esta manera, cada documento puede representarse como un vector numérico dentro de un espacio multidimensional, lo que permite aplicar diferentes métodos matemáticos para comparar documentos entre sí.

Una vez que los documentos han sido representados mediante vectores TF-IDF, es posible calcular la similitud entre ellos utilizando diversas métricas de distancia o similitud. Una de las métricas más utilizadas en procesamiento de texto es la similitud del coseno (Cosine Similarity). Esta métrica mide el ángulo entre dos vectores dentro de un espacio vectorial. Si dos documentos tienen vectores muy similares, el ángulo entre ellos será pequeño y el

valor de la similitud será cercano a 1. Por el contrario, si los documentos son muy diferentes, el ángulo será mayor y la similitud se acercará a 0.

La similitud del coseno es particularmente útil en tareas de análisis de texto porque permite comparar documentos independientemente de su longitud. Esto significa que dos documentos pueden ser considerados similares si comparten una distribución de palabras similar, incluso si uno de ellos contiene más palabras que el otro. Por esta razón, esta métrica es ampliamente utilizada en sistemas de recuperación de información, recomendación de documentos y análisis de similitud entre textos (Aggarwal, 2018).

En el contexto de esta práctica, estas técnicas se utilizan para analizar dos colecciones de cuentos y determinar cuáles historias presentan mayor similitud entre sí. Primero se realiza el preprocesamiento de los textos para limpiar y normalizar la información contenida en los documentos. Posteriormente, cada cuento es representado mediante vectores TF-IDF que reflejan la importancia de las palabras dentro de cada historia. Finalmente, se calcula la similitud entre los vectores de los cuentos pertenecientes a ambos libros utilizando la métrica de similitud del coseno. Este procedimiento permite identificar automáticamente cuáles historias comparten características lingüísticas similares y cuáles presentan mayor relación en términos de contenido textual.

Material y Equipo

Para la realización de esta práctica se utilizaron diferentes herramientas de software orientadas al procesamiento de lenguaje natural y análisis de datos. En primer lugar, se utilizó el lenguaje de programación Python, el cual es ampliamente utilizado en aplicaciones de ciencia de datos, inteligencia artificial y procesamiento de lenguaje natural debido a su facilidad de uso y a la gran cantidad de bibliotecas especializadas disponibles.

El desarrollo del programa se llevó a cabo utilizando diversas bibliotecas de Python. Una de las principales fue NLTK (Natural Language Toolkit), una biblioteca especializada en procesamiento de lenguaje natural que proporciona herramientas para el manejo y análisis de texto, incluyendo funciones para eliminación de *stopwords* y lematización de palabras. También se utilizó la biblioteca scikit-learn, la cual ofrece herramientas de aprendizaje automático y permite implementar técnicas de representación de texto como TF-IDF y métodos de cálculo de similitud entre documentos, como la similitud del coseno.

Además, se empleó la biblioteca pandas, utilizada para la manipulación y organización de datos en estructuras tipo tabla, lo que permitió generar y visualizar los resultados de similitud entre los cuentos de ambos libros de forma estructurada. También se utilizó el módulo re de Python para realizar operaciones de limpieza y procesamiento de texto mediante expresiones regulares.

En cuanto al hardware, la práctica fue realizada en una computadora personal con sistema operativo Windows, utilizando un entorno de desarrollo para ejecutar los scripts en Python y analizar los resultados generados por el programa.

Los archivos de texto utilizados como entrada del programa corresponden a dos colecciones de cuentos en formato digital, los cuales fueron procesados para identificar las historias individuales y posteriormente aplicar las técnicas de representación de texto y cálculo de similitud.

Desarrollo de la Practica

En esta práctica se desarrolló un programa en Python para analizar dos colecciones de cuentos y determinar cuáles historias presentan mayor similitud entre sí. Para lograrlo se implementó un proceso que incluye la lectura de archivos, limpieza del texto, preprocesamiento del lenguaje natural, representación de documentos mediante TF-IDF y cálculo de similitud utilizando la métrica de similitud del coseno.

1. Importación de librerías

En primer lugar se importaron las librerías necesarias para el procesamiento del texto y el cálculo de similitud entre documentos.

```
import re
import nltk
import pandas as pd
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

Figura 1: Librerías a utilizar

Las bibliotecas utilizadas cumplen las siguientes funciones:

- **re**: permite utilizar expresiones regulares para limpiar y procesar texto.
- **nltk**: proporciona herramientas para el procesamiento de lenguaje natural, como stopwords y lematización.
- **pandas**: se utiliza para organizar los resultados en forma de tabla.
- **scikit-learn**: permite aplicar técnicas de representación de texto como TF-IDF y calcular similitud entre vectores.

Posteriormente se descargaron algunos recursos necesarios de NLTK.

```
nltk.download("stopwords")
nltk.download("wordnet")
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()
```

Figura 2: Descarga de las stopwords y wordnet

Esto permite eliminar palabras comunes del idioma inglés y realizar el proceso de lematización.

2. Limpieza del texto

Antes de analizar los textos fue necesario eliminar información que no pertenece al contenido de los cuentos, como ilustraciones, encabezados o secciones del índice.

```
def remove_contents(text):
    if "CONTENTS" in text:
        text = text.split("CONTENTS",1)[1]
    return text
```

Figura 3: Remover contenidos

Esta función elimina la sección de tabla de contenidos, que aparece al inicio del libro y podría generar ruido en el análisis.

Posteriormente se implementó una función más general de limpieza.

```
def clean_text(text):
    text = remove_contents(text)
    text = re.sub(r"\[Illustration.*?\]", " ", text)
    text = re.sub(r"_Heading_", " ", text)
    text = re.sub(r"_Full-page design_", " ", text)
    text = re.sub(r"_Half-page design_", " ", text)
    text = re.sub(r"\s+", " ", text)
    return text
```

Figura 4: Limpieza de contenidos de Ilustración

En esta función se utilizan expresiones regulares para eliminar patrones específicos del texto que no pertenecen al contenido de los cuentos.

3. Separación de cuentos

Cada libro contiene múltiples historias dentro de un mismo archivo de texto. Para analizarlas individualmente fue necesario separar cada cuento utilizando sus títulos.

Primero se identifican las posiciones de los títulos dentro del texto.

```python
def find_story_positions(text, titles):
    positions = []
    for title in titles:
        pattern = re.compile(re.escape(title) + r"\.", re.IGNORECASE)
        match = pattern.search(text)
        if match:
            positions.append((title, match.start()))
    positions = sorted(positions, key=lambda x: x[1])
    return positions
```

Figura 5: Función de identificación de posiciones usando los títulos en el texto

Esta función busca cada título dentro del texto y guarda su posición. Posteriormente las posiciones se ordenan para identificar correctamente el inicio de cada cuento.

Después se utiliza esa información para extraer el texto correspondiente a cada historia.

```python
def split_stories(text, titles):
    text = clean_text(text)
    positions = find_story_positions(text, titles)
    stories = {}
    for i in range(len(positions)):
        title, start = positions[i]
        if i < len(positions)-1:
            end = positions[i+1][1]
        else:
            end = len(text)
        story = text[start:end]
        story = cut_story_end(story)
        stories[title] = story.strip()
    return stories
```

Figura 6: Función para separar historias y devolverlas

Esta función recorre todas las posiciones de los títulos y extrae el contenido que se encuentra entre un título y el siguiente.

4. Preprocesamiento del texto

Después de separar los cuentos se aplicó un proceso de preprocesamiento de lenguaje natural para preparar los textos para el análisis.

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)
    tokens = text.split()
    tokens = [lemmatizer.lemmatize(w) for w in tokens if w not in stop_words]
    return " ".join(tokens)
```

Figura 7: Función de preprocesamiento con 5 procesos importantes

Este proceso incluye:

- Conversión del texto a minúsculas
- Eliminación de caracteres especiales
- Tokenización (división del texto en palabras)
- Eliminación de stopwords
- Lematización de palabras

Estas transformaciones permiten obtener una representación más limpia y consistente del texto.

5. Representación de documentos con TF-IDF

Una vez preprocesados los cuentos, cada historia fue convertida en un vector numérico utilizando la técnica TF-IDF.

```
docs1 = [preprocess(x) for x in stories1.values()]
docs2 = [preprocess(x) for x in stories2.values()]
docs = docs1 + docs2
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs)
```

Figura 8. Vectorización usando TF-IDF

Cada cuento se representa como un vector donde cada dimensión corresponde a una palabra del vocabulario del corpus y su valor indica la importancia del término en el documento.

Posteriormente se separaron los vectores correspondientes a cada libro.

```
A = X[:len(docs1)]
B = X[len(docs1):]
```

Figura 9: Separación de vectores por cuento

6. Cálculo de similitud entre cuentos

Para comparar los cuentos de ambos libros se utilizó la métrica Cosine Similarity.

```
sim = cosine_similarity(A,B)
```

Figura 10. Métrica de Cosine Similarity

Esta función calcula la similitud entre cada cuento del primer libro y cada cuento del segundo libro, generando una matriz de similitud.

7. Generación de la tabla de resultados

Finalmente se generó una tabla con los resultados utilizando la biblioteca pandas.

```
results = []
titles1 = list(stories1.keys())
titles2 = list(stories2.keys())
for i in range(len(titles1)):
    for j in range(len(titles2)):
        results.append({
            "Story from Book 1": titles1[i],
            "Story from Book 2": titles2[j],
            "Similarity Score": sim[i,j]
        })
df = pd.DataFrame(results)
top5 = df.sort_values("Similarity Score", ascending=False).head(5)
print("\nTop 5 Most Similar Stories\n")
print(top5)
```

Figura 11. Impresión de tabla de similitud

Esta tabla contiene todas las combinaciones posibles entre cuentos de ambos libros y el valor de similitud correspondiente.

Finalmente se seleccionaron los cinco pares de cuentos con mayor similitud, que representan las historias más similares entre los dos libros analizados.

Resultados obtenidos

Después de aplicar las técnicas de procesamiento de lenguaje natural y calcular la similitud entre los cuentos de ambos libros, se generó una tabla con todas las combinaciones posibles de cuentos entre los dos conjuntos de textos. Cada fila de la tabla contiene el nombre de un cuento del primer libro, el nombre de un cuento del segundo libro y el valor de similitud calculado mediante la métrica Cosine Similarity aplicada sobre la representación TF-IDF de los textos.

Posteriormente, los resultados se ordenaron de mayor a menor según el valor de similitud para identificar cuáles pares de cuentos presentan mayor relación en términos de vocabulario y contenido textual.

La Figura 12 muestra los cinco pares de cuentos con mayor similitud entre ambos libros.

```
------------------------------------------------------------

Top 5 Most Similar Stories

                  Story from Book 1                                Story from Book 2  Similarity Score
452                   THE WHITE CAT                     THE PRINCE WITH THE GOLDEN HAND          0.355451
450                   THE WHITE CAT  THE HISTORY OF PRINCE SLUGOBYL; OR, THE INVISI...          0.311672
10   THE SLEEPING BEAUTY IN THE WOOD                     THE PRINCE WITH THE GOLDEN HAND          0.302955
571           THE HIND OF THE FOREST                     THE PRINCE WITH THE GOLDEN HAND          0.300922
454                   THE WHITE CAT                                             OHNIVAK          0.290462
```

Figura 12: Top 5 similitudes de ambos libros de cuentos

Los valores de similitud obtenidos se encuentran en un rango entre 0 y 1, donde valores cercanos a 1 indican que los documentos comparten una mayor cantidad de términos relevantes dentro de su representación vectorial.

En este caso, los resultados muestran que el cuento "THE WHITE CAT" presenta una alta similitud con varios cuentos del segundo libro, lo cual sugiere que estas historias comparten vocabulario relacionado con temas similares, como reinos, princesas, magia y personajes fantásticos. De manera similar, el cuento "THE PRINCE WITH THE GOLDEN HAND" aparece repetidamente en los resultados con diferentes cuentos del primer libro, lo que indica que contiene términos comunes dentro del género narrativo de cuentos de hadas.

Estos resultados demuestran que la técnica TF-IDF combinada con la similitud del coseno permite identificar automáticamente relaciones entre documentos basándose en el uso del lenguaje y el contenido textual.

Conclusiones y recomendaciones

En esta práctica se aplicaron técnicas fundamentales de Procesamiento de Lenguaje Natural (NLP) para analizar y comparar textos pertenecientes a dos colecciones de cuentos. A través del proceso de limpieza del texto, tokenización, eliminación de *stopwords* y lematización, fue posible preparar los documentos para su análisis computacional y reducir el ruido presente en los textos originales.

Posteriormente se utilizó la técnica de representación de texto TF-IDF, la cual permitió transformar cada cuento en un vector numérico que refleja la importancia de las palabras dentro de cada documento. Esta representación facilitó la aplicación de métodos matemáticos para comparar los textos entre sí. Mediante el uso de la similitud del coseno, fue posible calcular el grado de similitud entre los cuentos de ambos libros y generar una tabla con los cinco pares de historias más similares.

Durante el desarrollo de la práctica fue necesario realizar varios experimentos y pruebas adicionales para asegurar que los resultados obtenidos fueran correctos. Por ejemplo, se implementó una función que permite imprimir información de cada cuento, mostrando el título, número de tokens, número de caracteres, así como fragmentos del inicio y del final del texto. Esta verificación permitió comprobar que los cuentos se estaban separando correctamente dentro de los archivos originales.

También se agregó una impresión del número total de cuentos detectados en cada libro, lo cual permitió validar que el proceso de separación de historias estaba funcionando adecuadamente y que el programa estaba identificando correctamente todos los cuentos presentes en los textos. Estas verificaciones fueron importantes para detectar errores en las primeras versiones del código, como fragmentos de texto incorrectos o historias incompletas.

Otra observación importante durante el desarrollo de la práctica fue la necesidad de mejorar el proceso de limpieza del texto, ya que los archivos originales contenían elementos adicionales como ilustraciones, encabezados, marcas editoriales y secciones del proyecto Gutenberg que podían afectar el análisis. Para solucionar esto se implementaron expresiones regulares que eliminaron estos elementos y se agregó un mecanismo para cortar los cuentos en la marca "THE END.", evitando que información externa al cuento afectara los resultados de similitud.

Estos experimentos permitieron comprender la importancia de realizar una correcta preparación y validación de los datos antes de aplicar técnicas de análisis de texto. Además, ayudaron a garantizar que el modelo de representación y el cálculo de similitud se aplicaran únicamente sobre el contenido real de los cuentos.

Como recomendación para trabajos futuros, sería posible mejorar el análisis utilizando técnicas más avanzadas de representación de texto, como Word Embeddings (Word2Vec,

GloVe o BERT), que permiten capturar relaciones semánticas más profundas entre las palabras. También sería interesante aplicar estas técnicas a colecciones de textos más grandes para observar cómo se comportan los métodos de similitud en corpus de mayor tamaño.

Finalmente, esta práctica permitió comprender de manera práctica cómo las técnicas de procesamiento de lenguaje natural pueden aplicarse para analizar grandes volúmenes de texto y encontrar relaciones entre documentos de forma automática, así como la importancia del preprocesamiento y la validación de los datos para obtener resultados confiables.

Bibliografía

- Aggarwal, C. C. (2018). *Machine Learning for Text*. Springer International Publishing.
- Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media.
- Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed.). Stanford University. Disponible en: https://web.stanford.edu/~jurafsky/slp3/
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
- NLTK Project. (2024). *Natural Language Toolkit Documentation*. Disponible en: https://www.nltk.org/